arXiv:2402.02716v1 [cs.AI] 5 Feb 2024

Understanding the planning of LLM agents: survey

Xu Huang , **Weiwen Liu**[2] , **Xiaolong Chen** , **Xingmei Wang** , **Hao Wang** , **Defu Lian** , **Yasheng Wang**[2] , **Ruiming Tang**[2] , **Enhong Chen**

University of Science and Technology of China, Hefei, China
[2]Huawei Noah's rk Lab, Shenzhen, China
{xuhuangcs, chenxiaolong, xingmeiwang}@mail.ustc.edu.cn, {haowang, liandefu, cheneh}@ustc.edu.cn, {liuweiwen8, wangyasheng, tangruiming}@huawei.com

bstract

s Large Language Models (LLMs) have shown significant intelligence, the progress to leverage LLMs as planning modules of autonomous agents has attracted more attention. This survey provides the first systematic view of LLM-based agents planning, covering recent works aiming to improve planning ability. We provide a taxonomy of existing works on LLM- gent planning, which can be categorized into *Task Decomposition*, *Plan Selection*, *External Module*, *Reflection* and *Memory*. Comprehensive analyses are conducted for each direction, and further challenges for the field of research are discussed.

1 Introduction

utonomous agents have been recognized as intelligent entities capable of accomplishing specific tasks, via *perceiving the environment*, *planning*, and *executing actions*. Planning, as one of the most critical capabilities for agents, requires complicated understanding, reasoning, and decision-making progress [Ghallab *et al.*, 2004].

Despite the abstract concept of planning, a general formulation of the planning tasks can be described as follows. Given time step t, with the environment denoted as E, the action space as , the task goal as g, and the action at step t as a , the planning procedure can be expressed as the generation of a sequence of actions:

$$p = (a_0, a_1, \quad , a_t) = \text{plan}(E, g; \quad , \mathcal{P})$$

where and $\mathcal{P}$ represent the parameters of the LLM and the prompts for the task, respectively.

Conventional works mainly rely on symbolic methods or reinforcement learning-based methods, such as Planning Domain Definition Language (PDDL) [eronautiques *et al.*, 1998; Haslum *et al.*, 2019] or policy learning [He *et al.*, 2015; Yao *et al.*, 2020a]. However, those conventional methods have several limitations. Symbolic methods require conversion from flexible natural language-described problems into symbolic modeling, which may require human experts' efforts. Usually, this kind of method lacks error tolerance, resulting in failures even if there are only a few errors. Reinforcement learning (RL) methods are often combined with deep models, which serve as the policy network or reward model. While RL algorithms often require a large number of samples (interactions with the environment) to learn an effective policy, this can be impractical or costly in scenarios where collecting data is time-consuming or expensive.

Defu Lian is the corresponding author.

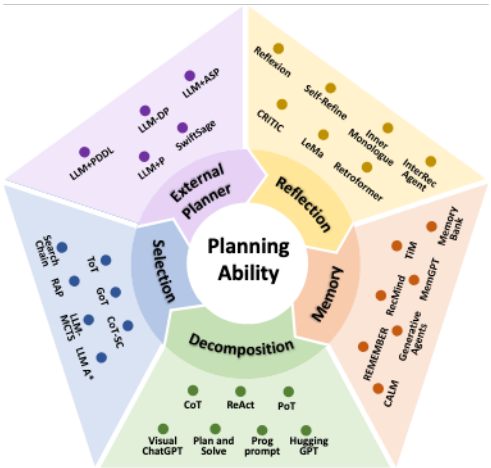

Figure 1: Taxonomy on LLM- gent planning.

In recent years, the emergence of Large Language Models (LLMs) has marked a paradigm shift. LLMs have achieved remarkable success across various domains, showcasing significant intelligence in reasoning, tool usage, planning, and instruction-following. The surprising intelligence of LLMs sheds light on employing LLMs as the cognitive core of agents, thereby offering the potential to improve planning ability. Numerous methodologies have been developed to harness the potential of LLMs for agent planning. While existing surveys have attempted to summarize techniques for LLMs [Zhao *et al.*, 2023a], LLMs for decision-making [Yang *et al.*, 2023a], reasoning [Sun *et al.*, 2023], tool learning [Qin *et al.*, 2023], and autonomous agents [Wang *et al.*, 2023a], they often lack a detailed analysis of planning ability within the literature. In this survey, we analyze the latest research works and discuss the advantages and limitations, aiming to provide a systematic view of the planning ability of LLM-based agents. Existing methods are further categorized into five representative directions, with each direction undergoing comprehensive analysis. Furthermore, we have evaluated

several representative methods on four benchmarks. To the best of our knowledge, this is the first work that comprehensively analyzes LLM-based agents from the planning abilities.

The subsequent sections of this paper are organized as follows. In Section 2, we categorize the works into five mainstream directions and analyze their ideas regarding planning ability. Sections 3 to 7 provide detailed discussions and analysis of each direction. Finally, Section 9 concludes the survey, offering insights into future directions in this field.

2 Taxonomy

s the research on the planning ability of LLM-based agents presents a flourishing scene, various methods have been proposed to exploit the upper limit of planning ability. To have a better bird's view of existing advanced works, we pick out some representative and influential works, analyzing their motivations and essential ideas. To provide a better understanding, we illustrate the analysis in Table 1.

ccording to the table, we present a novel and systematic taxonomy for LLM-based agent plannning that divides existing works into five important categories, covering *task decomposition*, *multi-plan selection*, *external module-aided planning*, *reflection and refinement* and *memory-augmented planning*, as illustrated in Figure 1. Here we briefly summarize those five directions as below.

Task Decomposition. Tasks in real life are usually complicated and multi-step, bringing severe hardness for planning. This kind of method adopts the idea of *divide and conquer*, decomposing the complicated into several sub-tasks and then sequentially planning for each sub-task. The process could be formulated as follows:

$$g_0, g_1, \quad , g_n = \text{decompose}(E, g; \quad , \mathcal{P});$$
$$p \ = (a_0, a_1, \quad a_m) = \text{sub-plan}(E, g \ ; \quad , \mathcal{P})$$

Multi-plan Selection. This kind of method focuses on leading the LLM to "think" more, generating various alternative plans for a task. Then a task-related search algorithm is employed to select one plan to execute. The process could be formulated as follows:

$$P = p_1, p_2, \quad , p_n = \text{plan}(E, g; \quad , \mathcal{P});$$
$$p \ = \text{select}(E, g, P; \quad , \mathcal{F})$$

where $\mathcal{F}$ represents the search strategies, such as some tree search algorithms [Yao *et al.*, 2023; Zhao *et al.*, 2023b].

External Planner- ided Planning. This methodology is crafted to employ an external planner to elevate the planning procedure, aiming to address the issues of efficiency and infeasibility of generated plans, while the LLM mainly plays the role in formalizing the tasks. The process could be formulated as follows:

$$h = \text{formalize}(E, g; \quad , \mathcal{P});$$
$$p = \text{plan}(E, g, h; \Phi)$$

where Φ denotes the external planner module, h represents the formalized information.

Reflection and Refinement. This methodology emphasizes improving planning ability through reflection and refinement. It encourages LLM to reflect on failures and then refine the plan. The process could be formulated as follows:

$$p_0 = \text{plan}(E, g; \quad , \mathcal{P});$$
$$r \ = \text{reflect}(E, g, p \ ; \quad , \mathcal{P});$$
$$p \quad _1 = \text{refine}(E, g, p \ , r \ ; \quad , \mathcal{P});$$

Memory-augmented Planning. This kind of approach enhances planning with an extra memory module, in which valuable information is stored, such as commonsense knowledge, past experiences, domain-specific knowledge, et al. The information is retrieved when planning, serving as auxiliary signals. The process could be formulated as follows:

$$m = \text{retrieve}(E, g; \mathcal{M});$$
$$p = \text{plan}(E, g, m; \quad , \mathcal{P})$$

where $\mathcal{M}$ represents the memory module.

The five directions are interconnected rather than mutually exclusive, often involving the concurrent adoption of multiple techniques. In the subsequent sections, we delve deeper into the five research directions concerning LLM-agent planning, elucidating their motivations, proposing representation solutions, and addressing inherent limitations.

3 Task Decomposition

In real-world scenarios, environments are often characterized by complexity and variability, thereby addressing complex tasks through a one-step planning process is a formidable challenge.

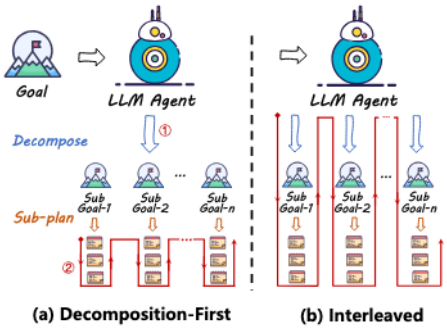

Figure 2: Types of task decomposition manners.

This simplification of complicated tasks is a remarkable human ability, evident in the decomposition of one task into several simpler sub-tasks [Schraagen *et al.*, 2000], which is analogous to the well-known algorithmic strategy called "divide and conquer", as illustrated in Eq. (1). Task decomposition generally involves two crucial steps: firstly, decomposing the complex task, referred to as the "decompose" step, and secondly, planning for the sub-tasks, known as the "sub-plan step". Current methods for task decomposition in this domain generally fall into two categories: decomposition-first and interleaved decomposition, illustrated in Figure 2.

3.1 Decomposition-First Methods

Decomposition-first methods decompose the task into subgoals first and then plan for each sub-goal successively, presented in Figure 2(a). The representative methods include

Table 1: taxonomy for existing LLM- gent planning works.

Method	Idea	LLM's task	Formulation	Representative works
Task Decomposition	Divide and Conquer	Task decomposition Subtask planning	$[g\ ] = \text{decompose}(E, g; \quad, \mathcal{P})$; $p\ = \text{sub-plan}(E, g\ ; \quad, \mathcal{P})$	CoT [2022], Re ct [2022], HuggingGPT [2023]
Multi-plan Selection	Generate multiple plans and select the optimal	Plans generation Plans evaluation	$P = \text{plan}(E, g; \quad, \mathcal{P})$; $p\ = \text{select}(E, g, P; \quad, \mathcal{F})$	ToT [2023], GoT [2023], CoT-SC [2022b]
External Planner-aided	Formalize tasks and utilize external planner	Task formalization	$h = \text{formalize}(E, g; \quad, \mathcal{P})$; $p = \text{plan}(E, g, h; \Phi)$	LLM+P [2023a], LLM+PDDL [2023]
Reflection & Refinement	Reflect on experiences and refine plans	Plan generation Reflection Refinement	$p_0 = \text{plan}(E, g; \quad, \mathcal{P})$; $r\ = \text{reflect}(E, g, p\ ; \quad, \mathcal{P})$; $p\ \ _1 = \text{refine}(E, g, p\ , r\ ; \quad, \mathcal{P})$	Reflexion [2023], CRITIC [2023], Self-Refine [2023]
Memory-aided Planning	Leverage memory to aid planning	Plan generation Memory extraction	$m = \text{retrieve}(E, g; \mathcal{M})$; $p = \text{plan}(E, g, m; \quad, \mathcal{P})$	REMEMBER [2023a], MemoryBank [2023]

HuggingGPT [Shen *et al.*, 2023], Plan-and-Solve [Wang *et al.*, 2023b], ProgPrompt [Singh *et al.*, 2023], et al.

HuggingGPT [Shen *et al.*, 2023] utilizes various multi-modal models from the Huggingface Hub to construct an intelligent agent for multimodal tasks. It is capable of handling tasks such as image generation, image classification, object recognition, video annotation, speech-to-text, et al. To facilitate collaboration between different models, the LLM acts as a controller, responsible for decomposing tasks inputted by humans, selecting models, and generating final responses. The most crucial stage is the initial task decomposition, where HuggingGPT explicitly instructs the LLM to break down the given task into sub-tasks, providing dependencies between tasks. Plan-and-Solve [Wang *et al.*, 2023b] improves upon the Zero-shot Chain-of-Thought [Kojima *et al.*, 2022] by transforming the original "*Let's think step-by-step*" into a two-step prompt instruction: "*Let's first devise a plan*" and "*Let's carry out the plan*". This zero-shot approach has achieved improvements in mathematical reasoning, common-sense reasoning, and symbolic reasoning. ProgPrompt [Singh *et al.*, 2023] translates natural language descriptions of tasks into coding problems. It symbolizes the agent's action space and objects in the environment through code, with each action formalized as a function and each object represented as a variable. Consequently, task planning is naturally transformed into function generation. When executing tasks, the agent first generates a plan in the form of function callings and then executes them step by step.

3.2 Interleaved Decomposition Methods

Interleaved decomposition involves interleaved task decomposition and sub-task planning, where each decomposition only reveals one or two sub-tasks at the current state, illustrated in Figure 2(b). Representative methods in this category include the Chain-of-Thought (CoT) series [Wei *et al.*, 2022; Kojima *et al.*, 2022], Re ct [Yao *et al.*, 2022], P L [Gao *et al.*, 2023], Program-of-Thought (PoT) [Chen *et al.*, 2022], Visual ChatGPT [Wu *et al.*, 2023], et al.

The introduction of Chain-of-Thought (CoT) [Wei *et al.*, 2022] reveals the few-shot learning capabilities of LLM. CoT guides the LLM in reasoning about complex problems through a few constructed trajectories, leveraging the LLM's reasoning abilities for task decomposition. Subsequently, Zero-shot CoT [Kojima *et al.*, 2022] unlocks the LLM's zero-shot reasoning abilities with the magical instruction "*Let's think step-by-step*". In contrast to CoT, which embeds reasoning within the planning process, Re ct [Yao *et al.*, 2022] decouples reasoning and planning. It alternates between reasoning (Thought step) and planning (ction step), demonstrating significant improvements in the planning capabilities. Visual ChatGPT [Wu *et al.*, 2023] utilizes Re ct's mechanism, employing LLM as the agent's brain equipped with a series of visual models, resulting in an agent with image processing capabilities. P L [Gao *et al.*, 2023] improves CoT by leveraging the LLM's coding abilities, guiding the LLM to generate code during reasoning. Finally, a code interpreter (such as Python) is used to comprehensively execute the codes to obtain the solution. This method proves helpful for agents in solving mathematical and symbolic reasoning problems. Program-of-Thought (PoT) [Chen *et al.*, 2022] completely formalize the reasoning process as programming. The authors also leverage a CodeX [Chen *et al.*, 2021b] model trained on code-related data, enhancing performance in mathematical and financial problems.

3.3 Discussions

For the decomposition-first method, the advantage lies in creating a stronger correlation between the sub-tasks and the original tasks, reducing the risk of task forgetting and hallucinations [Touvron *et al.*, 2023]. However, since the sub-tasks are predetermined at the beginning, additional mechanisms for adjustment are required otherwise one error in some step will result in failure, which will be discussed in Section 6. On the other hand, interleaved decomposition and sub-planning dynamically adjust decomposition based on environmental feedback, improving the fault tolerance. However, for complicated tasks, excessively long trajectories may lead to LLM experiencing hallucinations, deviating from the original goals during subsequent sub-tasks and sub-planning.

lthough task decomposition significantly enhances the ability of LLM- gent to solve complicated tasks, challenges persist. The first challenge is the additional overhead introduced by task decomposition. Decomposing a task into multiple sub-tasks requires more reasoning and generation,

incurring additional time and computational costs. On the other hand, for highly complex tasks that are decomposed into dozens of sub-tasks, the planning is constrained by the context length of the LLM, leading to the forgetting of the planning trajectories.

4 Multi-Plan Selection

Due to the complexity of the tasks and the inherent uncertainty of LLM, the plans generated by the LLM- gent for a given task can be diverse. Even though LLM possesses strong reasoning abilities, a single plan generated by LLM is likely to be suboptimal or even infeasible. more natural approach is multi-plan selection, comprising two major steps: multi-plan generation and optimal plan selection.

4.1 Multi-Plan Generation

Multi-plan generation involves generating a dozen paths of plans to comprise the candidate plan set. Mainstream methods consider employing uncertainty in the decoding process of generative models.

Self-consistency [Wang *et al.*, 2022b] employs a simple intuition: the solutions for complex problems are rarely unique. In contrast to CoT, which generates a single path, Self-consistency obtains multiple distinct reasoning paths via sampling strategies embodied in the decoding process, such as temperature sampling, top-k sampling. Tree-of-Thought (ToT) [Yao *et al.*, 2023] proposes two strategies to generate plans (i.e. thoughts): sample and propose. The *sample* strategy is consistent with Self-consistency, where LLM would sample multiple plans in decoding process. The *propose* strategy explicitly instructs the LLM to generate various plans via few-shot examples in prompts. Graph-of-Thought (GoT) [Besta *et al.*, 2023] extends ToT by adding transformations of thoughts, which supports arbitrary thoughts aggregation. LLM-MCTS [Zhao *et al.*, 2023b] and R P [Hao *et al.*, 2023] leverages LLM as the heuristic policy function for the Monte Carlo Tree Search (MCTS), where multiple potential actions are obtained by multiple calls.

4.2 Optimal Plan Selection

To select the optimal plan among the candidate plans, diverse strategies are adopted as heuristic search algorithms.

Self-consistency [Wang *et al.*, 2022b] applies the naive majority vote strategy, regarding the plan with the most votes as the optimal choice. Benefiting from the tree architecture, Tree-of-Thought (ToT) [Yao *et al.*, 2023] supports tree search algorithms, such as conventional BFS and DFS. When selecting a node for expansion, it uses LLM to evaluate multiple actions and chooses the optimal one. Similar with ToT, LLM-MCTS [Zhao *et al.*, 2023b] and R P [Hao *et al.*, 2023] also employ a tree structure to assist in multi-plan search. Unlike ToT, they employ the Monte Carlo Tree Search (MCTS) algorithm for search. LLM * [Xiao and Wang, 2023] utilizes the classic * algorithm from artificial intelligence to assist LLM in search. The Chebyshev distance from the current position to the target position serves as the heuristic cost function for selecting the optimal path.

4.3 Discussions

The scalability of multi-plan selection is notably advantageous, providing a broader exploration of potential solutions in the expansive search space. However, this advantage comes with inherent trade-offs. The increased computational demands, especially for models with large token counts or computations, pose practical challenges. This cost consideration becomes crucial, particularly in scenarios where resource constraints are a significant factor, such as the online service. Moreover, the reliance on LLM for the evaluation of plans introduces new challenges. s LLM's performance in ranking tasks is still under scrutiny, there is a need for further validation and fine-tuning of its capabilities in this specific context. The stochastic nature of LLMs adds randomness to the selection, potentially affecting the consistency and reliability of the chosen plans.

5 External Planner- ided Planning

Despite the powerful reasoning and task decomposition capabilities exhibited by Large Language Models (LLMs), challenges arise when confronted with environments featuring intricate constraints, such as mathematical problem-solving or generating admissible actions. To address challenges, several methods integrate LLMs with external planners. Such methods can be categorized into symbolic planners and neural planners based on the introduced planners.

5.1 Symbolic Planner

Symbolic planners have served as a fundamental component in the fields of automated planning for several decades. These approaches, based on well-established symbolic formalized models, such as PDDL models [eronautiques *et al.*, 1998; Haslum *et al.*, 2019], employ symbolic reasoning to identify optimal paths from initial states to desired goal states.

LLM+P [Liu *et al.*, 2023a] enhances the planning proficiency of LLMs by incorporating a PDDL-based symbolic planner. Leveraging the semantic understanding and coding capabilities of LLM, the authors organize problems into textual language prompts inputted to LLM. This prompts LLM to organize the actions within the environment and specified tasks into the format of the PDDL language. Subsequently, after obtaining a formalized description, the authors employ the Fast-Downward [1] solver for the planning process. Building upon LLM+P, LLM-DP [Dagan *et al.*, 2023] is specifically designed for dynamic interactive environments. Upon receiving feedback from the environment, LLM processes the information, formalizes it into PDDL language, and then employs a BFS [Lipovetzky *et al.*, 2014] solver to generate a plan. LLM+PDDL [Guan *et al.*, 2023] also utilizes the PDDL language to formalize the task, incorporating an additional step for manual verification to check for potential issues in the PDDL model generated by LLM. During the planning process, the authors propose using the plan generated by LLM as an initial heuristic solution to accelerate the search process of local search planners, such as LPG [Gerevini and Serina, 2002]. LLM+ SP [Yang *et al.*, 2023b] transforms problems described in natural language by LLM into atomic facts,

[1] https://github.com/aibasel/downward/tree/release-22.12.0

converting tasks into SP problems. Subsequently, the SP solver CLINGO is utilized to generate plans.

5.2 Neural Planner

Neural planners are deep models trained on collected planning data with reinforcement learning or imitation learning techniques, showing effective planning abilities within the specific domain. For instance, DRRN [He *et al.*, 2015] models the planning process as a Markov Decision Process through reinforcement learning, training a policy network to obtain a deep decision model. Decision Transformer (DT) [Chen *et al.*, 2021a] empowers a transformer model to clone human decision-making behavior with planning data.

Well-trained neural planners exhibit excellent planning capabilities within their respective domains and demonstrate superior planning efficiency due to their smaller parameter sizes. However, when faced with complex and less frequently encountered problems, where training data is scarce, these small models tend to perform poorly due to insufficient generalization ability. Therefore, several works explore combining an LLM with a light-weight neural planner, to further enhance the planning capabilities. C LM [Yao *et al.*, 2020a] proposed an early approach that combines a language model with an RL-based neural planner. One language model processes textual environmental information, generating a set of candidate actions as priors based on the environmental information. DRRN policy network is then employed to re-rank these candidate actions, ultimately selecting the optimal action. SwiftSage [Lin *et al.*, 2023] leverages the dual-process theory from cognitive psychology, dividing the planning process into slow thinking and fast thinking. The slow-thinking process involves complex reasoning and rational deliberation while fast-thinking resembles an instinctive response developed through long-term training. The authors utilize a DT model, trained through imitation learning, as the fast-thinking model for rapid plan generation. When errors occur during plan execution, indicating a more complex problem, the agent switches to the slow-thinking process, where LLM engages in reasoning and planning based on the current state. This combination of fast and slow thinking has proven to be highly effective in terms of efficiency.

5.3 Discussions

For those strategies that leverage an additional planner for assistance, LLM primarily plays a supportive role. Its main functions involve parsing textual feedback and providing additional reasoning information to assist in planning, particularly when addressing complex problems. Specifically, the enhancement of LLM's capabilities in code generation empowers the potential to deal with more general tasks for symbolic artificial intelligence. ctually, a significant drawback of traditional symbolic I systems lies in the complexity and heavy reliance on human experts in constructing symbolic models, while LLM accelerates this process, facilitating faster and more optimal establishment of symbolic models. The advantages brought by symbolic systems include theoretical completeness, stability, and interpretability. The combination of statistical I with LLM is poised to become a major trend in the future development of artificial intelligence.

6 Reflection and Refinement

Reflection and refinement are indispensable components in the planning process. They enhance the fault tolerance and error correction capabilities of LLM- gent planning. Due to existing hallucination issues and insufficient reasoning abilities for complex problems, LLM- gents may make errors and get stuck in "thought loops" during planning due to limited feedback. Reflecting on and summarizing failures helps agents correct errors and break out of such loops in subsequent attempts.

Self-refine [Madaan *et al.*, 2023] utilizes an iterative process of generation, feedback, and refinement. fter each generation, LLM generates feedback for the plan, facilitating adjustments based on the feedback. Reflexion [Shinn *et al.*, 2023] extends Re ct by incorporating an evaluator to assess trajectories. LLM generates self-reflections upon error detection, aiding in error correction. CRITIC [Gou *et al.*, 2023] uses external tools like Knowledge Bases and Search Engines to validate LLM-generated actions. It then leverages external knowledge for self-correction, significantly reducing factual errors. InteRec gent [Huang *et al.*, 2023b] employs a mechanism called ReChain for self-correction. n LLM is used to evaluate the response and tool-using plan generated by the interactive recommendation agent, summarize feedback on errors, and decide whether to restart planning. LEM [n *et al.*, 2023] gathers mistaken planning samples first and employs more powerful GPT-4 for correction. Those corrected samples are then used to fine-tune the LLM- gent, resulting in significant performance improvements across various scales of the LLaM model.

Particularly, the self-reflective strategy bears resemblance to the principles of reinforcement learning, where the agent plays the role of the decision-maker, such as the policy network. Environmental feedback triggers updates of the policy network. However, in contrast to deep reinforcement learning where updates are achieved by modifying model parameters, in the LLM agent, this update occurs through self-reflection by the LLM itself, culminating in textual verbal feedbacks. These textual feedbacks can serve as both long-term and short-term memory, influencing the agent's subsequent planning outputs through the prompts. Nevertheless, the convergence of this textual form of update currently lacks a guaranteed proof, indicating the inability to demonstrate that continual reflection can ultimately lead the LLM agent to a specified goal.

7 Memory- ugumented Planning

For agents, memory is a crucial pathway to enhance planning capabilities and the potential for growth. Regarding the memory mechanisms in LLM- gents, there are currently two major approaches to enhance planning abilities through memory: R G-based memory and embodied memory.

7.1 R G-based Memory

Retrieval ugmented Generation (R G) [Lewis *et al.*, 2020; Mao *et al.*, 2020; Cai *et al.*, 2022] techniques are proposed to aid text generation with retrieved information. It is capable of enhancing the LLM with the latest knowledge, such

as New Bing[2] and Google Bard[3]. For LLM agents, past experiences could be stored in the memory and retrieved when needed. The core idea of such methods is to retrieve task-relevant experiences from the memory during task planning. mong those methods, memories are typically stored in additional storage, and the forms are diverse, such as texts [Park *et al.*, 2023; Liu *et al.*, 2023b; Packer *et al.*, 2023; Wang *et al.*, 2023c; Zhong *et al.*, 2023], tabular forms [Zhang *et al.*, 2023a], knowledge graph [Pan *et al.*, 2024], etc.

Generative gents [Park *et al.*, 2023] store the daily experiences of human-like agents in text form and retrieve memories based on a composite score of recency and relevance to the current situation. Similarly, MemoryBank [Zhong *et al.*, 2023], TiM [Liu *et al.*, 2023b], and RecMind [Wang *et al.*, 2023c] encode each memory using a text encoding model into a vector and establish an indexing structure, such as F ISS library [Johnson *et al.*, 2019]. During retrieval, the description of the current status is used as a query to retrieve memories from the memory pool. The difference between the three lies in the way memories are updated. MemGPT [Packer *et al.*, 2023] leverages the concept of multiple levels of storage in computer architecture, abstracting the context of LLM into R M and treating the additional storage structure as a disk. LLM can spontaneously decide whether to retrieve historical memories or save the current context to storage. REMEMBER [Zhang *et al.*, 2023a] stores historical memories in the form of a Q-value table, where each record is (environment, task, action, Q-value)-tuple. During retrieval, positive and negative memories are both retrieved for LLM to generate plan based on the similarity of the environment and task.

7.2 Embodied Memory

Embodied memory involves finetuning the LLM with the agent's historical experiential samples, embedding memories into the model parameters. Usually the experiential samples are collected from the agents's interactions with environment, which may consist of commonsense knowledge about the environment, task-related priors, and successful or failed experiences. While the cost of training a language model with more than billions of parameters is huge, parameter-efficient fine-tuning (PEFT) techniques are leveraged to reduce cost and speed up by training a small part of parameters only, such as LoR , QLoR , P-tuning, et al.

C LM [Yao *et al.*, 2020b] utilizes ground-truth action trajectories collected from the text-world environment to fine-tune GPT-2 using next token prediction task, enabling it to memorize planning-related information and generalize well on planning tasks. Similarly, TDT [Wang *et al.*, 2022a] uses collected Markov decision process data to fine-tune Text Decision Transformer (TDT). It achieves better success rates on more challenging ScienceWorld [Wang *et al.*, 2022a] tasks. gentTuning [Zeng *et al.*, 2023] organizes plan trajectories from various tasks into a dialogue form to finetune the LLaM model, showing significant improvements in performance on unseen planning tasks.

[2]https://www.bing.com/

[3]https://bard.google.com

Table 2: Evaluation of representative prompt-based methods on four interactive benchmarks. The SR, R and EX are abbreviations of success rate, average rewards, and expenses respectively. The expenses are calculated based on the number of consumed tokens through Open I's PI. Z-CoT and F-CoT represent Zeroshot-CoT and Fewshot-CoT, respectively.

	lfWorld		ScienceWorld		HotPotQ		FEVER	
Metrics	SR(%)	EX($)	R	EX($)	SR(%)	EX($)	SR(%)	EX($)
Z-CoT	N/	N/	N/	N/	0.01	0.95	0.39	1.07
F-CoT	0.43	98.60	16.58	272.22	0.32	5.73	0.61	2.25
CoT-SC	0.57	105.37	15.24	274.33	0.33	7.86	0.62	3.21
SayCan	0.60	113.61	12.36	125.71	N/	N/	N/	N/
Re ct	0.57	152.18	15.05	356.03	0.34	66.00	0.63	22.20
Reflexion	0.71	220.17	19.39	724.48	0.39	112.49	0.68	37.26

7.3 Discussions

The R G-based and Fine-tuning-based memory approaches enhance LLM- gent planning capabilities, each with distinct advantages and limitations. R G-based methods offer real-time, low-cost external memory updates mainly in natural language text, but rely on the accuracy of retrieval algorithm. Finetuning provides a larger memorization capacity through parameter modifications but has high memory update costs and struggles with retaining fine-grained details.

Memory-enhanced LLM- gents demonstrate enhanced growth and fault tolerance in planning, yet memory generation heavily depends on LLM's generation capabilities. Improving weaker LLM- gents through self-generated memory remains a challenging area to explore.

8 Evaluation

Evaluating the planning capability of the agent is a critical issue in the research area. Here we investigate several mainstream benchmarking methods, categorizing them into the following types.

Interactive Gaming Environments: Game environments may provide real-time multi-modal feedback based on the agent's actions, including textual and visual feedback. Currently, the most widely used gaming environment is Minecraft [4], where the agent needs to gather materials to create tools for obtaining more rewards. The quantity of tools created by the agent is often used as an evaluation metric. nother popular category is the text-based interactive environments, such as LFWorld [Shridhar *et al.*, 2020], ScienceWorld [Wang *et al.*, 2022a], et al, where the agent locates in an environment described in natural language, with limited actions and locations. The success rate or the rewards obtained are commonly used as evaluation metrics. Compared with Minecraft, these text-based interactive environments are often simpler, with straightforward feedback and fewer feasible actions.

Interactive Retrieval Environments: Interactive retrieval environments simulate the process of information retrieval and reasoning that humans undergo in real life. In these environments, agents are often allowed to interact with search engines and other web services, using actions such as searching keywords or executing click, forward, and backward operations to acquire more information, thereby obtaining an-

[4]https://www.minecraft.net/

swers to questions or completing information retrieval tasks. Commonly used retrieval environments include question-answering tasks based on the Wikipedia engine [Yao *et al.*, 2022] (such as HotPotQ [Yang *et al.*, 2018] and Fever [Thorne *et al.*, 2018]) and web browsing tasks to find specific information, including WebShop, Mind2Web [Deng *et al.*, 2023], and Web rena [Zhou *et al.*, 2023]. The task success rate is usually used as the metric.

Interactive Programming Environments: Interactive programming environments simulate the interaction between programmers and computers, testing the agent's planning ability in solving computer-related problems. In these environments, agents are required to interact with computers to solve problems by writing code or instructions. They would receive various feedback including compile and runtime error messages, as well as execution results. Popular interactive programming environments involve issues related to operating systems, databases, etc., such as gent Bench [Liu *et al.*, 2023c], MiniWoB++ [Kim and others, 2023].

Most of these existing interactive environments lack fine-grained evaluation, where the performance is predominantly evaluated by the final success rate. Furthermore, unlike real-world scenarios where there are often multiple paths to complete a task, there is typically only one "golden" path in most simulated environments due to the high annotation cost.

Experiments. We have conducted experiments on four benchmarks to validate the performance of representative works, shown in Table 2. We have implemented six prompt-based methods due to limited budgets, covering task decomposition, multi-path selection, and reflection. s for the benchmarks, LFWorld, ScienceWorld, HotPotQ , and FEVER are employed, involving interactive gaming and question-answering benchmarks. Since LFWorld and ScienceWorld are involved in larger action space, the zero-shot method, i.e. ZeroShot-CoT, is not applicable due to unawareness of action space. SayCan improves CoT by grounding output actions into action space with a value function, which does not apply to Q tasks because there are only two actions: SE RCH[KEYWORD] and LOOKUP[KEYWORD]. nd we set the value function as a textual embedding model *bge-sm ll-en-v1.5* [Xiao and others, 2023]. We obtain 3 actions and 5 answers each step for gaming tasks and Q tasks for CoT-SC, respectively. The round of retries in Reflexion is set to 1. We use the PI of *text-d vinci-003* in Open I as LLM.

(i) The performance increases with the expenses. s CoT-SC, Re ct and Reflexion are involved in multiple plans, additional thoughts, and reflections, respectively, their expenses are more than their backbone methods. Intuitively, more tokens represent more detailed thinking, resulting in performance improvements.

(ii) Fewshot examples are suggested for complicated tasks. Despite that the magic instruction *Let's think step by step* can lead to more reasoning, ZeroShot-CoT exhibits severe performance degradation in two Q benchmarks, which demonstrates the necessity of the examples for LLM to further understand the task.

(iii) Reflection plays a crucial role in improving the success rate, especially for complex tasks. Despite Reflexion consuming about twice the tokens compared with Re ct, the improvements in complicated tasks are promising, such as LFWorld and ScienceWorld, which shows that LLM possesses the error-correcting capability.

9 Conclusions and Future Directions

Since LLM has shown the emergence of intelligence, there has been an increasing focus on using LLM to enhance the planning capabilities of agents. The major directions are summarized in Figure 1, with a detailed comparison and analysis of various methods presented in Sections 3 to 7. We also conducted experiments on four benchmarks, comparing the effectiveness of several representative methods and showing that performance increases with expenses. Despite the enhancements made by these works in planning capabilities, there are still some significant challenges.

Hallucinations. During the planning process, LLM often suffers from hallucinations, leading to irrational plans, unfaithfulness to task prompts, or failing to follow complex instructions. For instance, plans may include actions that interact with items not existed in the environment. lthough these issues can be alleviated through careful prompt engineering, they reflect fundamental shortcomings in LLM [Zhang *et al.*, 2023b; Huang *et al.*, 2023a].

Feasibility of Generated Plans. LLM, being fundamentally based on statistical learning, optimizes the probability of the next word through massive data. Compared to symbolic artificial intelligence, this approach struggles to obey complex constraints, especially when dealing with less common constraints encountered during LLM training. Consequently, plans generated by LLM may lack feasibility without considering adequate preconditions. Connecting LLM with symbolic planning models without altering LLM itself is a promising future direction.

Efficiency of Generated Plans. Generating efficient plans is a crucial issue in planning. However, in existing LLM agents, planning is greedily based on generated plans from LLM output, without considering the efficiency of the generated plans. Therefore, future developments may require introducing additional efficiency evaluation modules to work in conjunction with LLM for more efficient plans.

Multi-Modal Environment Feedback. LLM is originally designed for processing textual inputs, but real-world environment feedback is often multi-modal, including images, audio, etc., which are challenging to describe in natural language. Therefore, LLM agents face limitations when handling such scenarios. Future considerations may involve integrating the development of multi-modal large models and revisiting related planning strategies.

Fine-grained Evaluation. s mentioned in Section 8, existing benchmarks mostly rely on the final completion status of tasks, lacking fine-grained step-wise evaluations. dditionally, environmental feedback is often rule-based, simplistic, and distant from real-world scenarios. potential future direction is to leverage high-intelligence models like LLM to design more realistic evaluation environments.

References

[eronautiques *et al.*, 1998] Constructions eronautiques, dele Howe, et al. Pddl— the planning domain definition language. *Technical Report, Tech. Rep.*, 1998.

[n *et al.*, 2023] Shengnan n, Zexiong Ma, et al. Learning from mistakes makes llm better reasoner. *arXiv preprint arXiv:2310.20689*, 2023.

[Besta *et al.*, 2023] Maciej Besta, Nils Blach, et al. Graph of thoughts: Solving elaborate problems with large language models. *arXiv preprint arXiv:2308.09687*, 2023.

[Cai *et al.*, 2022] Deng Cai, Yan Wang, Lemao Liu, and Shuming Shi. Recent advances in retrieval-augmented text generation. In *SIGIR*, pages 3417–3419, 2022.

[Chen *et al.*, 2021a] Lili Chen, Kevin Lu, et al. Decision transformer: Reinforcement learning via sequence modeling. *NeurIPS*, 34:15084–15097, 2021.

[Chen *et al.*, 2021b] Mark Chen, Jerry Tworek, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[Chen *et al.*, 2022] Wenhu Chen, Xueguang Ma, et al. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.

[Dagan *et al.*, 2023] Gautier Dagan, Frank Keller, and lex Lascarides. Dynamic planning with a llm. *arXiv preprint arXiv:2308.06391*, 2023.

[Deng *et al.*, 2023] Xiang Deng, Yu Gu, et al. Mind2web: Towards a generalist agent for the web. *arXiv preprint arXiv:2306.06070*, 2023.

[Gao *et al.*, 2023] Luyu Gao, man Madaan, et al. Pal: Program-aided language models. In *ICML*, pages 10764–10799, 2023.

[Gerevini and Serina, 2002] lfonso Gerevini and Ivan Serina. Lpg: planner based on local search for planning graphs with action costs. In *ips*, volume 2, pages 281–290, 2002.

[Ghallab *et al.*, 2004] Malik Ghallab, Dana Nau, et al. *utomated Planning: theory and practice*. Elsevier, 2004.

[Gou *et al.*, 2023] Zhibin Gou, Zhihong Shao, et al. Critic: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint arXiv:2305.11738*, 2023.

[Guan *et al.*, 2023] Lin Guan, Karthik Valmeekam, et al. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. *arXiv preprint arXiv:2305.14909*, 2023.

[Hao *et al.*, 2023] Shibo Hao, Yi Gu, et al. Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992*, 2023.

[Haslum *et al.*, 2019] Patrik Haslum, Nir Lipovetzky, et al. *n introduction to the planning domain definition language*, volume 13. Springer, 2019.

[He *et al.*, 2015] Ji He, Jianshu Chen, et al. Deep reinforcement learning with a natural language action space. *arXiv preprint arXiv:1511.04636*, 2015.

[Huang *et al.*, 2023a] Lei Huang, Yu Weijiang, et al. survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.

[Huang *et al.*, 2023b] Xu Huang, Jianxun Lian, et al. Recommender ai agent: Integrating large language models for interactive recommendations. *arXiv preprint arXiv:2308.16505*, 2023.

[Johnson *et al.*, 2019] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[Kim and others, 2023] Geunwoo Kim et al. Language models can solve computer tasks. *arXiv preprint arXiv:2303.17491*, 2023.

[Kojima *et al.*, 2022] Takeshi Kojima, Shixiang Shane Gu, et al. Large language models are zero-shot reasoners. *NeurIPS*, 35:22199–22213, 2022.

[Lewis *et al.*, 2020] Patrick Lewis, Ethan Perez, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *NeurIPS*, 33:9459–9474, 2020.

[Lin *et al.*, 2023] Bill Yuchen Lin, Yicheng Fu, et al. Swiftsage: generative agent with fast and slow thinking for complex interactive tasks. *arXiv preprint arXiv:2305.17390*, 2023.

[Lipovetzky *et al.*, 2014] Nir Lipovetzky, Miquel Ramirez, et al. Width and inference based planners: Siw, bfs (f), and probe. *IPC*, page 43, 2014.

[Liu *et al.*, 2023a] Bo Liu, Yuqian Jiang, et al. Llm+ p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.

[Liu *et al.*, 2023b] Lei Liu, Xiaoyan Yang, et al. Think-in-memory: Recalling and post-thinking enable llms with long-term memory. *arXiv preprint arXiv:2311.08719*, 2023.

[Liu *et al.*, 2023c] Xiao Liu, Hao Yu, et al. gent-bench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.

[Madaan *et al.*, 2023] man Madaan, Niket Tandon, , et al. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.

[Mao *et al.*, 2020] Yuning Mao, Pengcheng He, Liu, et al. Generation-augmented retrieval for open-domain question answering. *arXiv preprint arXiv:2009.08553*, 2020.

[Packer *et al.*, 2023] Charles Packer, Vivian Fang, et al. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[Pan *et al.*, 2024] Shirui Pan, Linhao Luo, et al. Unifying large language models and knowledge graphs: roadmap. *TKDE*, 2024.

[Park *et al.*, 2023] Joon Sung Park, Joseph O'Brien, et al. Generative agents: Interactive simulacra of human behavior. In *SUIST*, pages 1–22, 2023.

[Qin *et al.*, 2023] Yujia Qin, Shengding Hu, et al. Tool learning with foundation models. *arXiv preprint arXiv:2304.08354*, 2023.

[Schraagen *et al.*, 2000] Jan Maarten Schraagen, Susan F Chipman, et al. *Cognitive task analysis*. Psychology Press, 2000.

[Shen *et al.*, 2023] Yongliang Shen, Kaitao Song, et al. Hugginggpt: Solving ai tasks with chatgpt and its friends in huggingface. *arXiv preprint arXiv:2303.17580*, 2023.

[Shinn *et al.*, 2023] Noah Shinn, Federico Cassano, et al. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

[Shridhar *et al.*, 2020] Mohit Shridhar, Xingdi Yuan, et al. lfworld: ligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768*, 2020.

[Singh *et al.*, 2023] Ishika Singh, Valts Blukis, et al. Progprompt: Generating situated robot task plans using large language models. In *ICR 2023*, pages 11523–11530. IEEE, 2023.

[Sun *et al.*, 2023] Jiankai Sun, Chuanyang Zheng, et al. survey of reasoning with foundation models. *arXiv preprint arXiv:2312.11562*, 2023.

[Thorne *et al.*, 2018] James Thorne, ndreas Vlachos, et al. Fever: a large-scale dataset for fact extraction and verification. *arXiv preprint arXiv:1803.05355*, 2018.

[Touvron *et al.*, 2023] Hugo Touvron, Louis Martin, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[Wang *et al.*, 2022a] Ruoyao Wang, Peter Jansen, et al. Scienceworld: Is your agent smarter than a 5th grader? *arXiv preprint arXiv:2203.07540*, 2022.

[Wang *et al.*, 2022b] Xuezhi Wang, Jason Wei, et al. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

[Wang *et al.*, 2023a] Lei Wang, Chen Ma, et al. survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2023.

[Wang *et al.*, 2023b] Lei Wang, Wanyu Xu, et al. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *arXiv preprint arXiv:2305.04091*, 2023.

[Wang *et al.*, 2023c] Yancheng Wang, Ziyan Jiang, et al. Recmind: Large language model powered agent for recommendation. *arXiv preprint arXiv:2308.14296*, 2023.

[Wei *et al.*, 2022] Jason Wei, Xuezhi Wang, et al. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 35:24824–24837, 2022.

[Wu *et al.*, 2023] Chenfei Wu, Shengming Yin, et al. Visual chatgpt: Talking, drawing and editing with visual foundation models. *arXiv preprint arXiv:2303.04671*, 2023.

[Xiao and others, 2023] Shitao Xiao et al. C-pack: Packaged resources to advance general chinese embedding, 2023.

[Xiao and Wang, 2023] Hengjia Xiao and Peng Wang. Llm a*: Human in the loop large language models enabled a* search for robotics. *arXiv preprint arXiv:2312.01797*, 2023.

[Yang *et al.*, 2018] Zhilin Yang, Peng Qi, et al. Hotpotqa: dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.

[Yang *et al.*, 2023a] Sherry Yang, Nachum Ofir, et al. Foundation models for decision making: Problems, methods, and opportunities. *arXiv preprint arXiv:2303.04129*, 2023.

[Yang *et al.*, 2023b] Zhun Yang, dam Ishay, and Joohyung Lee. Coupling large language models with logic programming for robust and general reasoning from text. *arXiv preprint arXiv:2307.07696*, 2023.

[Yao *et al.*, 2020a] Shunyu Yao, Rohan Rao, et al. Keep calm and explore: Language models for action generation in text-based games. *arXiv preprint arXiv:2010.02903*, 2020.

[Yao *et al.*, 2020b] Shunyu Yao, Rohan Rao, et al. Keep calm and explore: Language models for action generation in text-based games. *arXiv preprint arXiv:2010.02903*, 2020.

[Yao *et al.*, 2022] Shunyu Yao, Jeffrey Zhao, et al. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

[Yao *et al.*, 2023] Shunyu Yao, Dian Yu, et al. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023.

[Zeng *et al.*, 2023] ohan Zeng, Mingdao Liu, et al. gent-tuning: Enabling generalized agent abilities for llms. *arXiv preprint arXiv:2310.12823*, 2023.

[Zhang *et al.*, 2023a] Danyang Zhang, Lu Chen, et al. Large language model is semi-parametric reinforcement learning agent. *arXiv preprint arXiv:2306.07929*, 2023.

[Zhang *et al.*, 2023b] Yue Zhang, Yafu Li, et al. Siren's song in the ai ocean: survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219*, 2023.

[Zhao *et al.*, 2023a] Wayne Xin Zhao, Kun Zhou, et al. survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

[Zhao *et al.*, 2023b] Zirui Zhao, Wee Sun Lee, and David Hsu. Large language models as commonsense knowledge for large-scale task planning. *arXiv preprint arXiv:2305.14078*, 2023.

[Zhong *et al.*, 2023] Wanjun Zhong, Lianghong Guo, Qiqi Gao, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2023.

[Zhou *et al.*, 2023] Shuyan Zhou, Frank F Xu, et al. Webarena: realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.